

Refactorización de la clase `Variable`
Extracción de responsabilidades y consolidación del modelo de dominio
`org.openmarkov.core` — paquete `model.network`

Manuel Arias Calleja

Marzo de 2026

Índice

1. Introducción	2
2. Fundamentos teóricos	2
2.1. Principio de Responsabilidad Única (SRP)	2
2.2. Patrón <i>Extract Class</i>	2
2.3. ¿Qué es un acoplamiento?	3
3. Situación anterior: Variable con 986 líneas	3
3.1. Problemas identificados	4
3.1.1. 1. Violación del SRP: lógica de edición del grafo	5
3.1.2. 2. Inconsistencia name/baseName/timeSlice	5
3.1.3. 3. Orden de comparación no determinista	5
3.1.4. 4. Inicialización inconsistente de propiedades	5
3.1.5. 5. Método con error tipográfico	5
3.1.6. 6. Duplicación en potenciales delta	5
4. Refactorización aplicada	6
4.1. Paso 1: Ampliación de tests	6
4.2. Paso 2: Extracción de la lógica de edición de estados (<i>Extract Class</i>)	6
4.3. Paso 3: Consolidación de name/baseName/timeSlice	7
4.4. Paso 4: Corrección de <code>compareTo()</code>	8
4.5. Paso 5: Inicialización <i>eager</i> de <code>statesAdditionalProperties</code>	8
4.6. Paso 6: Eliminación de <code>checkNewStateName()</code> y unificación de potenciales delta	9
5. Resultado final	9
5.1. Resumen numérico	11
5.2. Ficheros afectados	11
6. Verificación	11
7. Conclusiones	11

1. Introducción

Este documento describe la refactorización de la clase `Variable` del módulo `org.openmarkov.core`, que pasó de **986 líneas** a **690 líneas** mediante una serie de mejoras incrementales.

La refactorización sigue el patrón *Extract Class* [1] y se fundamenta en el *Principio de Responsabilidad Única* (SRP) [2]. Cada concepto se explica en la sección 2 de forma accesible antes de describir los cambios concretos.

Todos los cambios se verificaron con una batería de **99 tests unitarios** (71 de ellos nuevos), más los 410 tests del módulo `core`, todos pasando sin errores.

2. Fundamentos teóricos

2.1. Principio de Responsabilidad Única (SRP)

El *Single Responsibility Principle* establece que **una clase debe tener una única razón para cambiar** [2]. Si una clase acumula múltiples responsabilidades, un cambio en una de ellas puede introducir errores en las demás.

Ejemplo intuitivo: pensemos en un documento de texto que también contuviera la lógica de la impresora y del corrector ortográfico. Si necesitáramos cambiar el formato de impresión, podríamos romper accidentalmente el corrector. En software, cada “responsabilidad” debería vivir en su propia clase.

Cuando una clase viola SRP, los síntomas típicos son:

- La clase tiene muchas líneas (cientos o miles).
- Los métodos se agrupan en bloques temáticos que no se relacionan entre sí.
- Los cambios en una parte de la clase obligan a re-testear partes no relacionadas.
- Es difícil nombrar la clase con un sustantivo único que describa todo lo que hace.

2.2. Patrón *Extract Class*

El patrón *Extract Class* [1] es la técnica de refactorización que se aplica cuando una clase tiene demasiadas responsabilidades. Consiste en:

1. Identificar un grupo cohesivo de métodos que forman una responsabilidad independiente.
2. Crear una nueva clase con esos métodos.
3. En la clase original, sustituir la lógica extraída por una **delegación** (una llamada a la nueva clase).

En nuestro caso, los métodos extraídos no usan campos privados de `Variable` —solo acceden a métodos públicos como `getStates()`, `setStates()`, `getVariableType()`—, lo que permite moverlos a una **clase de utilidad estática** que recibe la `Variable` como parámetro.

En nuestro caso concreto, los métodos extraídos se movieron a una **clase de utilidad estática**: una clase no instanciable que agrupa métodos `static` sin estado propio. Este patrón ya se usa en OpenMarkov; por ejemplo, `VariableTypeConverter` fue extraída previamente de la clase `Node` con el mismo enfoque.

2.3. ¿Qué es un acoplamiento?

Decimos que una clase A está **acoplada** a otra clase B cuando A necesita conocer la existencia de B para funcionar. Un acoplamiento alto dificulta la reutilización y el testing, porque para probar A hay que construir también B y todas sus dependencias.

En nuestro caso, **Variable** estaba acoplada a **Node**, **ProbNet**, **Link**, **Potential** y **UniformPotential** solo por los métodos de edición de estados. Al extraerlos, la clase **Variable** queda libre de esas dependencias del grafo.

3. Situación anterior: Variable con 986 líneas

La clase **Variable** representaba una variable aleatoria (o de decisión) en una red bayesiana. Un diagrama UML simplificado de la situación anterior se muestra en la Figura 1.

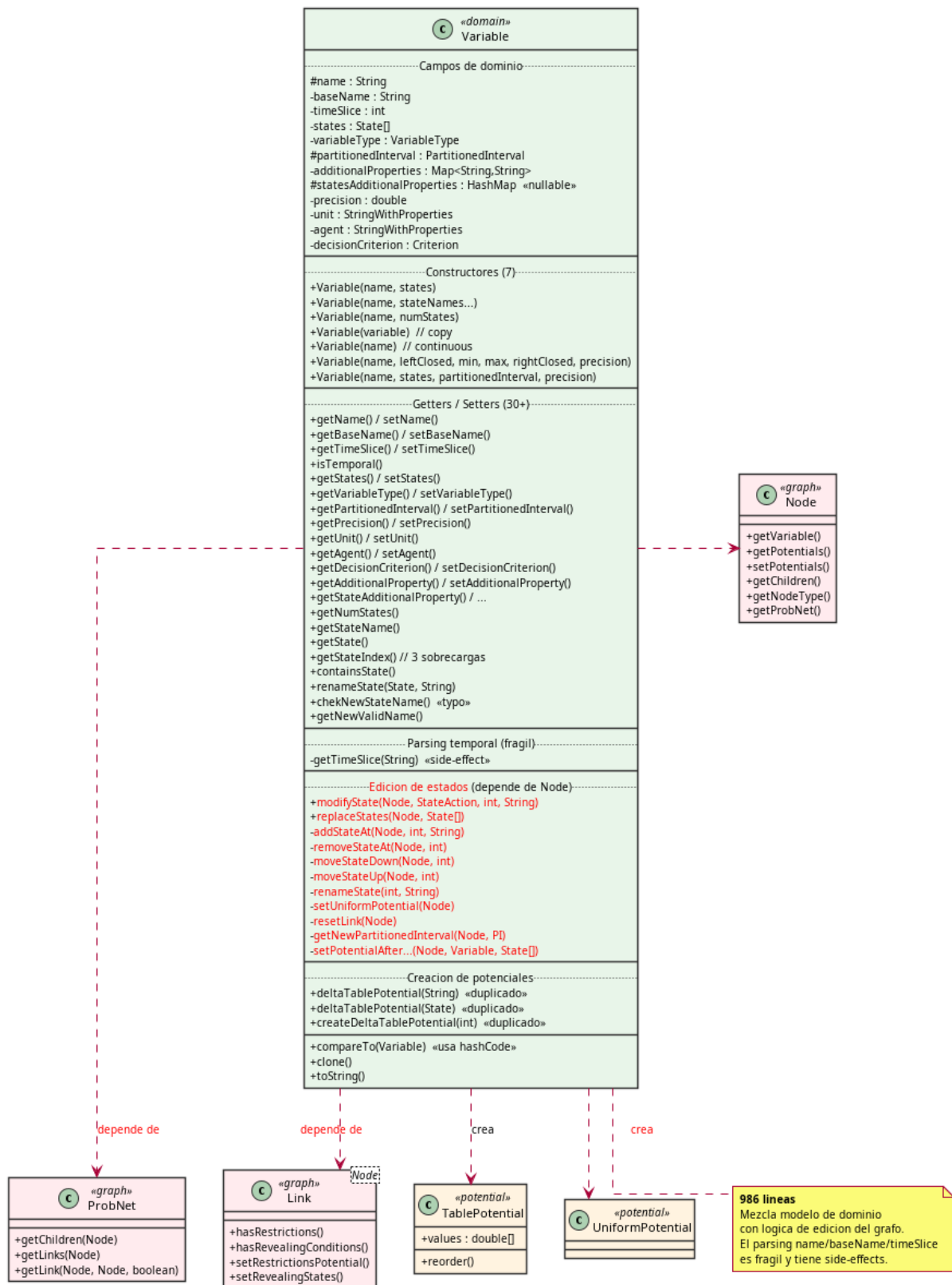


Figura 1: Diagrama UML de **Variable** antes de la refactorización. Los métodos en **rojo** son los que dependían de **Node/ProbNet/Link**.

3.1. Problemas identificados

Se identificaron seis problemas principales:

3.1.1. 1. Violación del SRP: lógica de edición del grafo

La clase mezclaba dos responsabilidades claramente distintas:

- **Modelo de dominio:** nombre, tipo, estados, intervalo, precisión — lo que describe a la variable en sí misma.
- **Edición del grafo:** métodos como `addStateAt()`, `removeStateAt()`, `setUniformPotential()`, `resetLink()` que modifican potenciales de nodos hijos, resetean restricciones de arcos, etc.

Estos últimos métodos recibían un `Node` como parámetro y operaban sobre `ProbNet`, `Link` y diversas subclases de `Potential`. Una variable no debería conocer la topología del grafo en el que participa.

En total, **10 métodos privados y 2 públicos** (unas **180 líneas**) pertenecían a esta responsabilidad ajena.

3.1.2. 2. Inconsistencia name/baseName/timeSlice

Las variables temporales (como “X [3]”) almacenan su nombre en tres campos interdependientes:

- **name:** el nombre completo (ej. “X [3]”).
- **baseName:** el nombre sin el índice temporal (ej. “X”).
- **timeSlice:** el índice temporal (ej. 3).

Existían **tres puntos distintos** donde estos campos se sincronizaban: `setName()`, `setBaseName()` y `setTimeSlice()`. Cada uno reimplementaba la lógica de parsing o reconstrucción de forma ligeramente distinta, y el método privado `getTimeSlice(String)` tenía un *side-effect*: modificaba el campo `baseName` como efecto secundario de parsear el nombre.

El propio código fuente reconocía el problema con un TODO:

```
1 // TODO mantener la consistencia entre name y baseName cuando se cambian
```

3.1.3. 3. Orden de comparación no determinista

El método `compareTo()` comparaba variables por su `hashCode()`, que en Java depende de la dirección de memoria del objeto. Esto produce un orden arbitrario que cambia entre ejecuciones del programa, lo que puede causar comportamientos no reproducibles cuando las variables se almacenan en colecciones ordenadas.

3.1.4. 4. Inicialización inconsistente de propiedades

El campo `additionalProperties` se inicializaba como `new HashMap<>()` en la declaración, pero `statesAdditionalProperties` se dejaba a `null`, lo que obligaba a incluir comprobaciones de nulidad en los cuatro métodos que lo usaban.

3.1.5. 5. Método con error tipográfico

El método `chekNewStateName()` (falta una “c” en *check*) era funcionalmente idéntico a `!containsState()` y no tenía ningún uso en código de producción fuera de la propia clase.

3.1.6. 6. Duplicación en potenciales delta

Tres métodos —`deltaTablePotential(String)`, `deltaTablePotential(State)` y `createDeltaTablePotential`— contenían lógica duplicada para crear potenciales delta (distribuciones donde toda la probabilidad

se concentra en un solo estado).

4. Refactorización aplicada

La refactorización se realizó en seis pasos incrementales, cada uno verificado con la batería completa de tests antes de pasar al siguiente.

4.1. Paso 1: Ampliación de tests

Antes de modificar la clase, se amplió la cobertura de tests de **28 tests** a **99 tests**, añadiendo pruebas para:

- Todos los constructores (con array de estados, varargs, numérico, continuo, discretizado, copia).
- Método `clone()`.
- Propiedades adicionales de variable y de estados.
- Variables temporales (parsing, `timeSlice`, `baseName`).
- Consultas de estados (`getState()`, `getStateIndex()`, `containsState()`).
- Potenciales delta.
- Intervalos por defecto y redondeo.
- Getters/setters de propiedades (`unit`, `agent`, `decisionCriterion`).

Estos tests actuaron como **red de seguridad**: si alguna refactorización posterior introducía una regresión, los tests la detectarían inmediatamente.

4.2. Paso 2: Extracción de la lógica de edición de estados (*Extract Class*)

Se creó la clase `VariableStateOperations` siguiendo el patrón de `VariableTypeConverter` ya existente en el proyecto.

Métodos extraídos (10 privados + 2 públicos):

Método	Responsabilidad
<code>modifyState()</code>	Dispatcher: ADD, REMOVE, UP, DOWN, RENAME
<code>replaceStates()</code>	Reemplazo completo de estados
<code>addStateAt()</code>	Añadir un estado en una posición
<code>removeStateAt()</code>	Eliminar un estado
<code>moveStateDown()</code>	Intercambiar estado con el anterior
<code>moveStateUp()</code>	Intercambiar estado con el siguiente
<code>renameState(int, String)</code>	Renombrar estado por índice
<code>setUniformPotential()</code>	Resetear potenciales a distribución uniforme
<code>resetLink()</code>	Limpiar restricciones de arcos
<code>getNewPartitionedInterval()</code>	Calcular nuevo intervalo tras añadir estado
<code>reorderStatesInNodeAndChildren()</code>	Reordenar potenciales tras mover estado
<code>setPotentialAfterReordering...</code>	Helper de reordenación

En `Variable`, los métodos públicos quedaron como delegaciones de una línea:

```
1 public void modifyState(Node node, StateAction action, int idx, String name) {  
2     VariableStateOperations.modifyState(this, node, action, idx, name);  
3 }
```

El flujo de la delegación se muestra en la Figura 2.

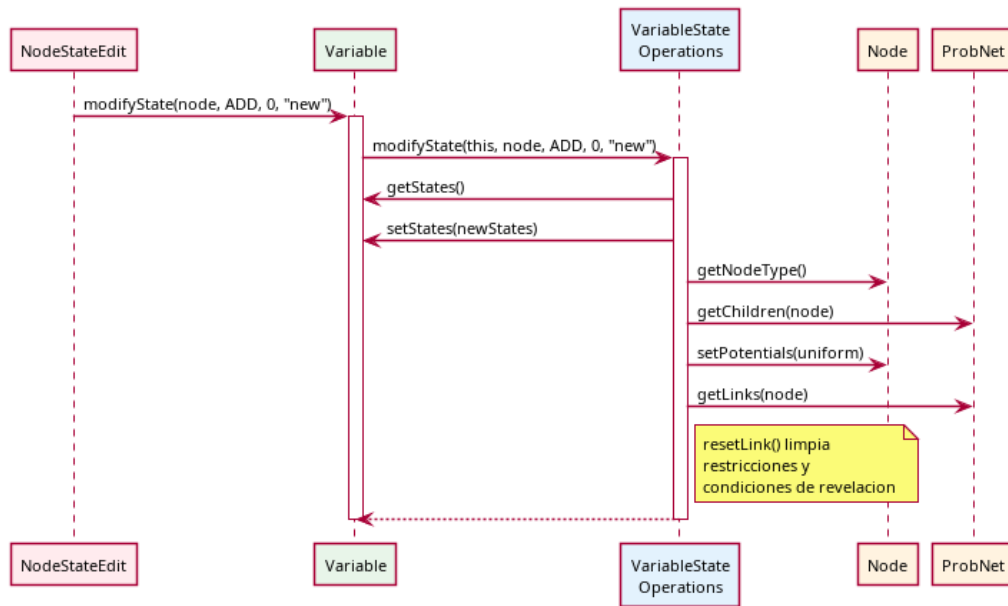


Figura 2: Diagrama de secuencia: flujo de una operación ADD tras la refactorización. **Variable** delega a **VariableStateOperations**, que es quien interactúa con **Node** y **ProbNet**.

4.3. Paso 3: Consolidación de name/baseName/timeSlice

Se reemplazaron los tres puntos de sincronización por dos métodos privados:

- **parseNameIntoParts(String)**: descompone un nombre completo como "X [3]" en **baseName**="X" y **timeSlice**=3. Es el único punto de *parsing*.
- **rebuildFullName()**: reconstruye **name** a partir de **baseName** y **timeSlice**. Es el único punto de *composición*.

Todos los constructores y setters usan ahora exclusivamente estos dos métodos, eliminando el TODO y garantizando la consistencia. La Figura 3 muestra la diferencia.

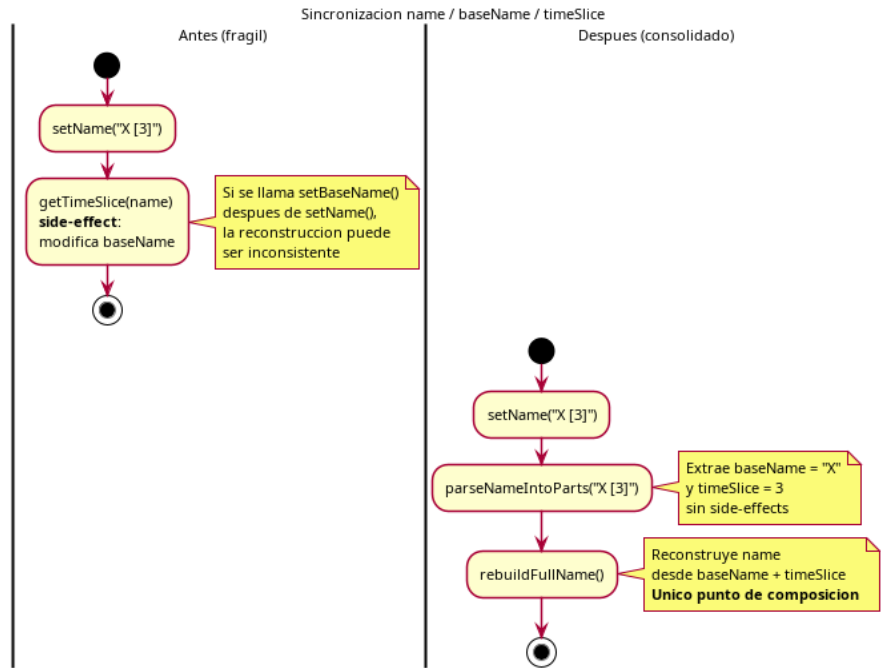


Figura 3: Antes y después de la consolidación del parsing temporal. Antes, `getTimeSlice(String)` tenía un side-effect sobre `baseName`. Después, hay un único flujo limpio.

4.4. Paso 4: Corrección de `compareTo()`

Se sustituyó la comparación por `hashCode()` —no determinista entre ejecuciones— por una comparación por `name` (orden alfabético):

```

1 // Antes (no determinista):
2 @Override public int compareTo(Variable o) {
3     int othersHashCode = o.hashCode();
4     int thisHashCode = hashCode();
5     // ...comparacion por hashCode...
6 }
7
8 // Despues (determinista):
9 @Override public int compareTo(Variable o) {
10     return this.name.compareTo(o.name);
11 }

```

4.5. Paso 5: Inicialización *eager* de `statesAdditionalProperties`

Se cambió la declaración del campo de:

```

1 protected HashMap<...> statesAdditionalProperties; // null inicial

```

a:

```

1 protected HashMap<...> statesAdditionalProperties = new HashMap<>();

```

Esto permitió eliminar las comprobaciones de nulidad en los cuatro métodos que lo usan y simplificar `setStateAdditionalProperty()` usando `computeIfAbsent()`:

```

1 public void setStateAdditionalProperty(String state, String key, String value)
2     {
3         statesAdditionalProperties
4             .computeIfAbsent(state, k -> new HashMap<>())
5             .put(key, value);
    }

```

4.6. Paso 6: Eliminación de `chekNewStateName()` y unificación de potenciales `delta`

- Se eliminó `chekNewStateName()` (sin usos en producción, equivalente a `!containsState()`).
- Las tres variantes de `deltaTablePotential` se unificaron: las versiones con `String` y `State` ahora delegan en `createDeltaTablePotential(int)`, eliminando la duplicación.

5. Resultado final

La Figura 4 muestra el diagrama UML de la clase `Variable` tras todas las refactorizaciones.

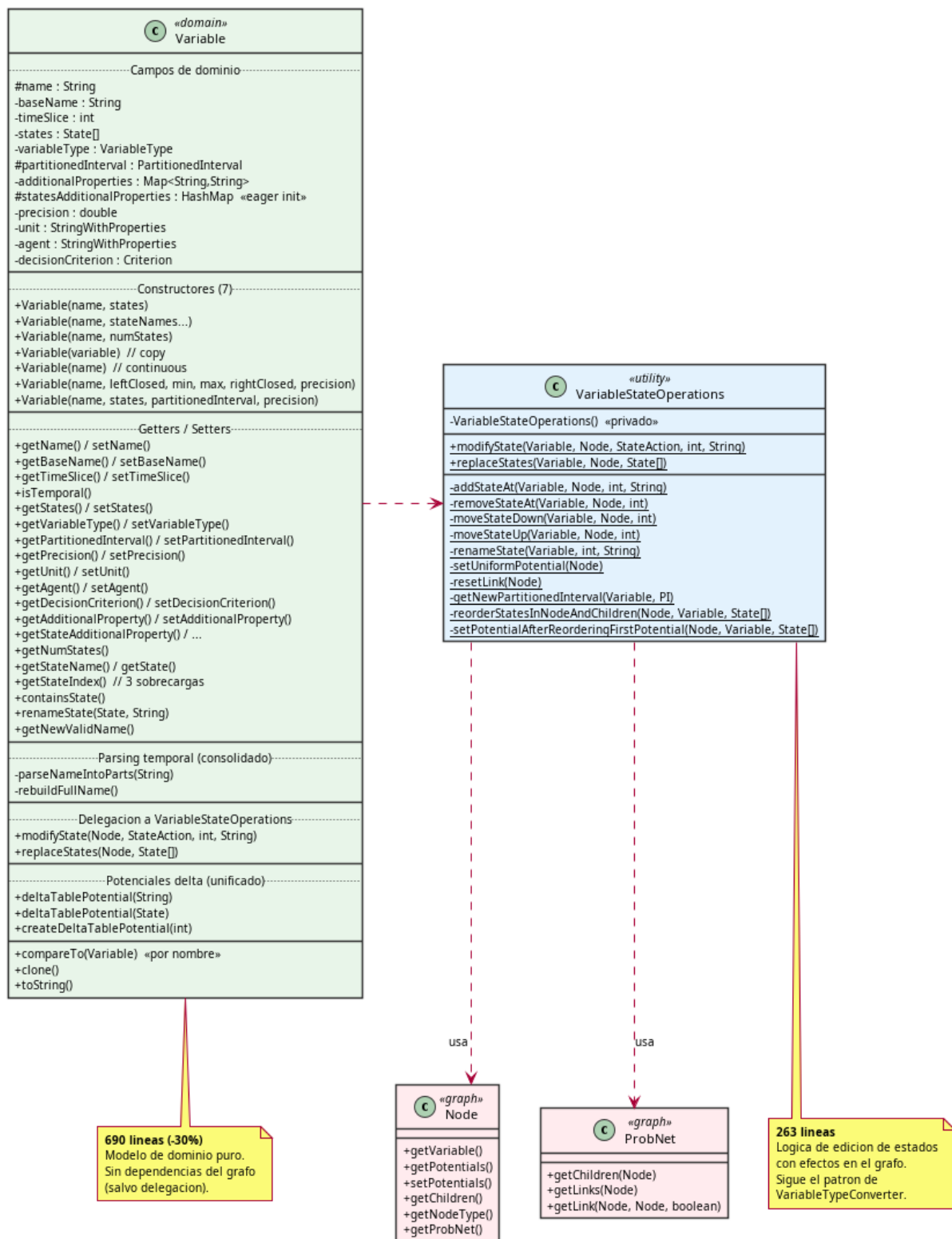


Figura 4: Diagrama UML de Variable y VariableStateOperations después de la refactorización. Variable es ahora un modelo de dominio puro.

5.1. Resumen numérico

Métrica	Antes	Después
Líneas de <code>Variable.java</code>	986	690 (-30 %)
Dependencias de <code>Variable</code> hacia el grafo	4 clases	0 (delegación)
Métodos públicos duplicados	3	0
Null checks en <code>statesAdditionalProperties</code>	5	0
Puntos de sincronización <code>name/baseName</code>	3	2 (centralizados)
Tests unitarios	28	99
Tests del módulo <code>core</code>	410	408 ¹
Tests fallidos	0	0

5.2. Ficheros afectados

Fichero	Cambio
<code>Variable.java</code>	Eliminados 10 métodos privados, consolidado parsing, simplificados null checks, eliminado método con typo, corregido <code>compareTo()</code> , unificados delta potentials
<code>VariableStateOperations.java</code>	Nuevo (263 líneas) — lógica de edición de estados extraída de <code>Variable</code>
<code>VariableTest.java</code>	Ampliado de 4 a 47 tests (más nested classes)
<code>VariableModifyStateTest.java</code>	Sin cambios (24 tests siguen pasando)
<code>NodeStateEdit.java</code>	Sin cambios
<code>NodeReplaceStatesEdit.java</code>	Sin cambios

6. Verificación

Cada paso de la refactorización se verificó ejecutando:

```
# Tests específicos de Variable
mvn test -pl org.openmarkov.core \
  -Dtest="VariableTest,VariableModifyStateTest"

# Todos los tests del modulo core
mvn test -pl org.openmarkov.core
```

En todos los pasos, los 408 tests del módulo `core` pasaron con 0 fallos y 0 errores.

7. Conclusiones

La refactorización de `Variable` logró:

1. **Separar responsabilidades:** la lógica de edición del grafo ahora reside en `VariableStateOperations`, y `Variable` es un modelo de dominio puro.
2. **Eliminar fragilidad:** la sincronización `name/baseName/timeSlice` se centralizó en dos métodos, eliminando el riesgo de inconsistencias.
3. **Mejorar la mantenibilidad:** código más corto, sin null checks innecesarios, sin duplicaciones, y con un orden de comparación determinista.
4. **Aumentar la confianza:** la batería de tests se multiplicó por 3,5, cubriendo áreas que antes no se probaban.

¹La diferencia de 2 tests se debe a la eliminación de los tests del método `checkNewStateName()`, que fue eliminado.

5. **Mantener compatibilidad:** no se modificó ninguna clase llamadora (`NodeStateEdit`, `NodeReplaceStatesEdit`), gracias al patrón de delegación.

La refactorización sigue el precedente establecido por la extracción previa de `VariableTypeConverter` desde `Node`, manteniendo la coherencia arquitectónica del proyecto.

Referencias

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2019.
- [2] R.C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.